Programação Funcional

Prática a06 - Funções de ordem superior

Ficha PL2 — Tarefa 5 a 10
Cap. 6, pp. 44–52

Da recursão a uma ideia nova

Nas aulas anteriores percorremos listas com recursão.

```haskell
dobrar :: Num a => [a] -> [a]

dobrar [] = []
dobrar (x:xs) = (2*x) : dobrar xs
```

E se quisermos calcular o quadrado de todos os números?

Será necessário escrever sempre a mesma estrutura recursiva?

O que muda entre estas duas funções?

```haskell
dobrar :: Num a => [a] -> [a]
dobrar [] = []
dobrar (x:xs) = (2*x) : dobrar xs
```

```haskell
quadrados :: Num a => [a] -> [a]
quadrados [] = []
quadrados (x:xs) = (x*x) : quadrados xs
```

A lógica muda? Ou muda apenas a operação que estamos a fazer?

A estrutura recursiva é semelhante. Só muda a operação aplicada a cada elemento.

Podemos reutilizar o mesmo padrão de processamento?

Funções de ordem superior

```haskell
aplica :: (a -> b) -> a -> b
-- aplica recebe uma função e um valor.
-- f é a função; x é o valor a que essa função vai ser aplicada.

aplica f x = f x

-- f é uma função.
-- Por exemplo, f pode ser (+1), even ou length.
-- x é o argumento que damos a essa função.
-- O resultado é o valor produzido por f x.

-- Exemplo:
-- aplica (+1) 3 = 4
```

Em Haskell, as funções podem receber outras funções como argumentos e podem devolver funções como resultado.

Aplicação de uma função como argumento

aplica (+3) 4

→ (+3) 4

→ 4 + 3

→ 7

```
aplica :: (a -> b) -> a -> b
aplica f x = f x
```

Porque é que o resultado de aplica even 4 é um Bool e não um número?

a e b podem ser de tipos diferentes

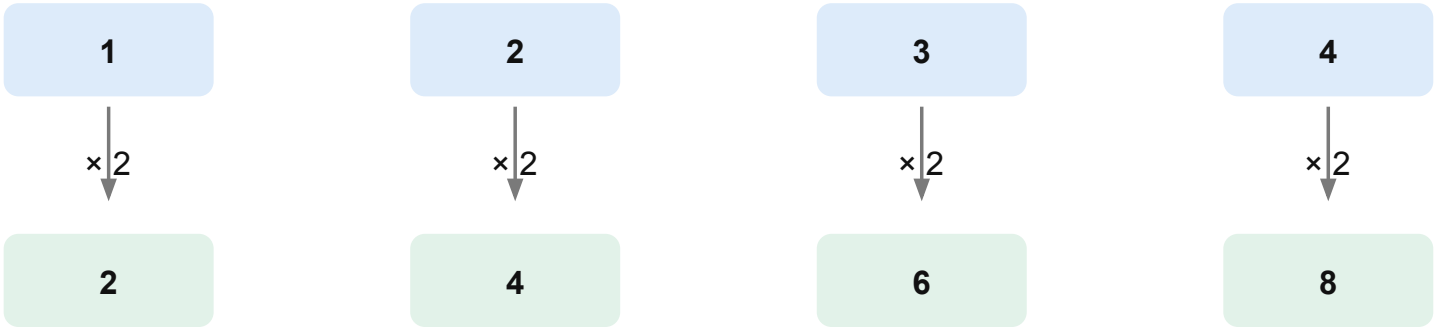
aplica even 4

True

map — Transformar elementos de uma lista

```
map :: (a -> b) -> [a] -> [b]
```

Aplica a mesma função a cada elemento da lista.



map (*2) [1,2,3,4] → [2,4,6,8]

map no GHCi — prevê antes de executar

```
map (*2) [1..10]

map (+3) [1,5,3,1,6]

map even [1,2,3,4]

map length ["a", "Haskell", "abc"]
```

Qual é o resultado de cada expressão?

Como funciona map por dentro?

```
map :: (a -> b) -> [a] -> [b]
map _ [] = []
map f (x:xs) = f x : map f xs
```

Exemplo de avaliação:

map (*2) [1,2,3]
→ 2 : map (*2) [2,3]
→ 2 : 4 : map (*2) [3]
→ 2 : 4 : 6 : []
→ [2,4,6]

Recursão explícita vs map

COM RECURSÃO EXPLÍCITA

```
dobrar :: Num a => [a] -> [a]
dobrar [] = []
dobrar (x:xs) =
  (2*x) : dobrar xs
```

COM MAP

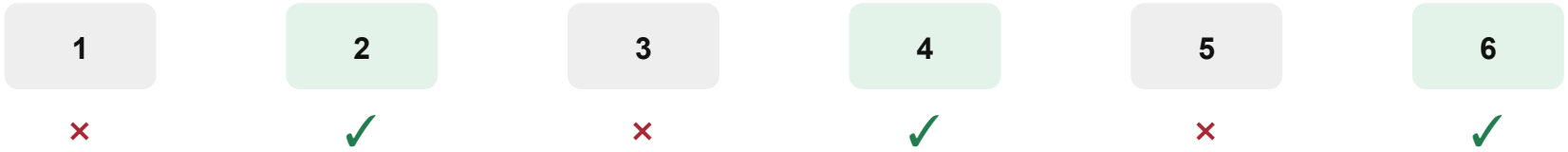
```
dobrar :: Num a => [a] -> [a]
dobrar = map (*2)
```

Deixamos de usar recursão, ou deixamos de a escrever?

filter — selecionar elementos

```
filter :: (a -> Bool) -> [a] -> [a]
```

Mantém os elementos para os quais o predicado devolve True.



filter even [1,2,3,4,5,6] → [2,4,6]

filter no GHCi — o que é conservado?

```
ghci> filter even [1..10]
[2,4,6,8,10]

ghci> filter (>3) [1,5,3,2,1,6,4]
[5,6,4]

ghci> filter (<0) [-3,4,-2,7,0]
[-3,-2]
```

O que fazem even, (>3) e (<0)?

Como funciona filter por dentro?

```
filter :: (a -> Bool) -> [a] -> [a]
filter _ [] = []
filter p (x:xs)
  | p x       = x : filter p xs
  | otherwise = filter p xs
```

Em cada passo:

- Testar se p x é True.
- Se for True, conservar x.
- Se for False, ignorar x e continuar.

Exercício orientado — positivos

Queremos obter apenas os números positivos de uma lista.

```
positivos :: (Num a, Ord a) => [a] -> [a]

positivos = ...
```

Exemplos:

```
ghci> positivos [-3,4,0,-2,7]
[4,7]

ghci> positivos []
[]
```

Devemos utilizar map ou filter?

```haskell
positivos :: (Num a, Ord a) => [a] -> [a]
-- Recebe e devolve listas com elementos do mesmo tipo a.
-- Num a permite usar 0; Ord a permite comparar com >.

positivos = filter (> 0)

-- (> 0) testa se um elemento é maior que zero.
-- filter conserva apenas os elementos que passam no teste.
-- Equivale a escrever: positivos xs = filter (> 0) xs
-- A ordem mantém-se; zero e negativos ficam de fora.

-- Exemplos:
-- positivos [-3,4,0,-2,7] = [4,7]
-- positivos [] = []
```

map e filter — não são a mesma coisa

MAP

```
map (*2) [1,2,3,4]
→ [2,4,6,8]
```

- Transforma elementos
- Conserva o comprimento da lista
- Pode alterar o tipo

FILTER

```
filter even [1,2,3,4]
→ [2,4]
```

- Seleciona elementos
- Pode diminuir o comprimento
- Preserva o tipo dos elementos

Lambdas — funções sem nome

Por vezes precisamos de uma função apenas uma vez.

```
quadrado x = x * x
```

equivale a

```
\x -> x * x
```

```
ghci> map (\x -> x * x) [1,2,3]
[1,4,9]
```

O símbolo \ inicia uma função anônima em Haskell.

Em \x -> x * x, Lê-se: «função que recebe x e devolve x multiplicado por x».

map + filter — podemos combinar as duas?

```
map (^2) (filter even [1,2,3,4])
```

Qual das funções seleciona e qual transforma?

1. Que lista produz filter even [1,2,3,4]?

2. O que acontece quando map (^2) recebe essa lista?

map + filter — avaliação passo a passo

```
map (^2) (filter even [1,2,3,4])
```



```
map (^2) [2,4]
```



```
[4,16]
```

Primeiro selecionamos; depois transformamos.

Exercício orientado — quadrados

Queremos calcular o quadrado de cada número de uma lista.

```
quadrados :: Num a => [a] -> [a]

quadrados = ...
```

Exemplos:

```
ghci> quadrados [-3,0,2,5]
[9,0,4,25]

ghci> quadrados []
[]
```

Que função e que lambda devemos utilizar?

```haskell
quadrados :: Num a => [a] -> [a]
-- Recebe e devolve listas com elementos do mesmo tipo a.
-- Num a permite multiplicar os valores com (*).

quadrados = map (\x -> x * x)

-- \ inicia uma função anónima (lambda).
-- x é o parâmetro; x * x calcula o seu quadrado.
-- map aplica esta função a cada elemento da lista.
-- A ordem e o número de elementos mantêm-se.

-- Exemplos:
-- quadrados [-3,0,2,5] = [9,0,4,25]
-- quadrados [] = []
```

Exercícios autónomos

Num ficheiro a06.hs, implementar e testar:

- duplicar — Recebe uma lista de números e devolve uma lista com o dobro de cada elemento, usando map.
- celsiusParaFahrenheit — Recebe temperaturas em Celsius e devolve-as em Fahrenheit. Usa F = C × 9/5 + 32.
- positivos — Recebe uma lista de números e devolve apenas os maiores que zero, usando filter.
- notasAprovadas — Recebe uma lista de notas e devolve apenas as notas iguais ou superiores a 10.
- quadradosPares — Recebe uma lista de inteiros e devolve os quadrados dos elementos pares, pela mesma ordem.

Para cada função: escrever a assinatura, testar com [] e justificar a escolha.

```haskell
quadradosPares :: Integral a => [a] -> [a]
-- Recebe inteiros e devolve os quadrados dos que são pares.
-- Integral a permite testar a paridade com even.

quadradosPares xs = map (\x -> x * x) (filter even xs)

-- xs é a lista de entrada.
-- filter even xs seleciona os números pares.
-- map (\x -> x * x) calcula o quadrado de cada um.
-- A ordem mantém-se; os números ímpares ficam de fora.

-- Exemplo: quadradosPares [-3,-2,0,4,5]
-- filter even [-3,-2,0,4,5] = [-2,0,4]
-- map (\x -> x * x) [-2,0,4] = [4,0,16]
-- Lista vazia: quadradosPares [] = []
```

Trabalho prático

Fase 1 — Modelação e núcleo funcional

O objetivo é construir as bases da aplicação, não a aplicação completa.

Moodle da UC > Framework para o Trabalho Prático

https://moodle.utad.pt/pluginfile.php/77516/mod_resource/content/1/Framework_para_Trabalho_Pratico_de_Programacao_Funcional.pdf

Fase 1 — o que têm de entregar?

- Modelo de domínio: data types, records e relações.
- Estrutura inicial: módulos e organização do repositório.
- Funções puras com regras essenciais do projeto.
- Funções recursivas adequadas ao problema.
- Dados de teste para demonstração.
- README com descrição e instruções.
- Histórico Git do trabalho do grupo.

Entrega 8 de novembro, 23h30

Código de exemplo — como representar um aluno?

```
data Aluno = Aluno
  { numero    :: Int
  , nome      :: String
  , nota1     :: Float
  , nota2     :: Float
  , notafinal :: Float
  } deriving (Show)
```

Aluno é o tipo de dados; o construtor Aluno cria um valor com cinco campos.

numero e nome identificam o aluno; os restantes campos guardam as notas.

deriving (Show) permite apresentar o valor como texto no GHCi.

Que informação precisamos de guardar para cada aluno?

Código de exemplo — uma pauta de alunos

```
type Pauta = [Aluno]

aluno1 :: Aluno
aluno1 = Aluno 123 "Ana" 14 16 15

pauta :: Pauta
pauta = [aluno1]
```

`Pauta` é um sinónimo de `[Aluno]`, não um novo tipo de dados.

`aluno1` representa a Ana: número 123, notas 14 e 16 e nota final 15.

`pauta` contém um único aluno. Uma pauta vazia representa-se por `[]`.

Fase 1 — as primeiras funções puras

```haskell
media :: Float -> Float -> Float
media a b = (a + b) / 2

aprovado :: Aluno -> Bool
aprovado aluno = notafinal aluno >= 10
```

media recebe dois valores Float e devolve a sua média, também Float.

aprovado recebe um Aluno e devolve Bool: True se a nota final for pelo menos 10.

notafinal aluno obtém a nota final do aluno.

São funções puras: dependem dos argumentos e não têm efeitos de IO.

Fase 1 — mostrar recursividade no projeto

```haskell
contarAprovados :: Pauta -> Int
contarAprovados [] = 0
contarAprovados (a:as)
  | notafinal a >= 10 = 1 + contarAprovados as
  | otherwise         = contarAprovados as
```

contarAprovados recebe uma Pauta e devolve a contagem como Int.

Caso base: [] tem zero aprovados.

Padrão (a:as): primeiro aluno e lista restante.

Guardas: soma 1 se a nota for pelo menos 10.

Recursão: continua a contar em as.

Código do exemplo — o que pertence a outra fase?

```haskell
-- Fase 2: funções de ordem superior
valida :: String -> String -> Bool
valida numero contents =
  null $ filter (encontra numero) (lines contents)

-- Fase 3: interação com o exterior
lerAlunos :: IO String
lerAlunos = readFile "alunos.txt"
guardarTexto :: String -> IO ()
guardarTexto texto = appendFile "alunos.txt" texto
```

valida devolve True se o número não existir no conteúdo recebido.

lines separa linhas; filter seleciona correspondências; null testa se não há nenhuma.

$ aplica null ao resultado à direita.

lerAlunos lê o ficheiro; guardarTexto acrescenta texto no fim.

Na Fase 1, o foco é a lógica sem IO.

Preparar a demonstração da Fase 1

Na apresentação, cada grupo deverá conseguir:

- Explicar o modelo de dados criado.
- Ler e justificar as assinaturas das funções.
- Apontar o caso base e a chamada recursiva.
- Demonstrar funções com dados reais e casos limite.
- Mostrar a organização do código no GitHub.

No GHCi: carregar o módulo, consultar Aluno e contar os aprovados nos dados de teste. (exemplo neste caso dos alunos)

```
:load Fase1.hs
:info Aluno
contarAprovados pautaTeste
```

Ficha PL2 - Tarefa 5 a 10

no git https://github.com/v2denny/pf2627

pasta >aulas>a06>FichaPL2.pdf

Desafio — qual versão preferes?

Resolver o mesmo problema de duas formas:

Recursão explícita **VS** **map / filter**

Qual é mais fácil de ler, reutilizar e testar?